

Software Engineering Group Project- Maintenance Manual

Author(s): Francy Sasso - frs30

Niall Perry - nip33

Config Ref: Maintenance-GP6

Date: 05/05/2025

Version: 1.0

Status: Release

Department of Computer Science

Aberystwyth University

Aberystwyth

Ceredigion

SY23 3DB

Copyright © Aberystwyth University 2025

Table of Contents

TABLE OF CONTENTS.....	2
1 INTRODUCTION	4
1.1 PURPOSE OF THIS DOCUMENT	4
1.2 SCOPE.....	4
1.3 OBJECTIVES.....	4
2 MANUAL.....	5
2.1 PROJECT CONFIGURATION.....	5
JAVA JDK	5
LANGUAGE LEVEL.....	5
JAVAFX.....	5
INTELLIJ IDE	5
MAVEN.....	6
JUNIT	6
2.2 PROGRAM DESCRIPTION.....	6
2.3 PROGRAM STRUCTURE	7
2.3.1 FLOW OF CONTROL DIAGRAM	7
2.3.2 COMPONENT DIAGRAM	8
2.3.3 SEQUENCE DIAGRAMS	8
2.3.4 UML CLASS DIAGRAMS	8
2.4 ALGORITHMS	9
DECK INITIALIZATION AND SHUFFLING	9
CARD DRAWING LOGIC	9
CREATION OF CARDS.....	9
EQUALITY CHECKING	9
RECORDING AND REPLAY	9
2.5 DATA STRUCTURES	10
CARD	10
DECK.....	10
REPLAY / RECORDING	10
ENUMS	10

2.6 FILES	11
CARD VISUAL FILES	11
RECORDING FILES	11
REPLAYING FILES.....	11
2.7 INTERFACES.....	12
2.8 SUGGESTIONS FOR IMPROVEMENTS	12
CONSERVING CARD ROTATION AFTER FLIPPING/REVEALING	12
DISABLE THE ABILITY TO DRAG CARDS OFF THE SCREEN	12
EXITING FULL SCREEN BUG	12
SORT OUT ISSUES WITH REPLAY	13
DEVELOPERS' SUGGESTIONS	13
2.9 MAKING CHANGES.....	14
TABLETOP PACKAGE.....	14
UI PACKAGE	14
UTIL PACKAGE.....	14
2.10 PHYSICAL LIMITATIONS	15
2.11 REBUILDING AND TESTING	15
3 REFERENCES.....	16
4 DOCUMENT HISTORY	16

1 Introduction

1.1 Purpose of this document

The purpose of this document is to provide the facilities for future project maintainers to understand its development process, properties, and design, to make effective and efficient changes to the source code and other components of the project.

1.2 Scope

This document should be read by all team members and any future developers or teams that intend to alter or improve upon the program's design or functioning in any way. It is assumed that the reader is already aware of and familiar with the following documents:

- SE.QA.RS-CS22120 - Deck of cards product backlog CS/CC22120 [1]
- SE.QA.04 - Ui prototype standards [2]
- SE.QA.05 - Design Standards for CS/CC22120 [3]
- SE.QA.06 - Test Procedure Standards [4]
- SE.QA.08 - Configuration Management standards [5]
- SE.QA.09 - Coding standards for CS/CC22120 [6]
- UI-Spec-Docu-GP6 - UI Specification Document and Presentation [7]
- Design-Spec-GP6 - Design Specification [8]
- Test-Spec-GP6 - Test Specification [9]

1.3 Objectives

The document will address the following goals:

1. Describing the program structure and its functioning.
2. Detailing program properties and configuration details.
3. Discuss algorithms, resources, and data structures used.
4. Future alterations, improvements, changes, etc.
5. Maintaining the program effectively

2 Manual

2.1 Project Configuration

Java JDK

The project was built and compiled using Eclipse Temurin 21.0.1 SDK (temurin-21). We chose to use this JDK due to its open-source build, TCK-certified (Java-compliant), and support of development workflow when using GitLab. Temurin-21 was also compatible with the user stories specified in SE.QA.RS-CS22120 [1]. It's licensed under the Eclipse Public License 2.0 (EPL-2.0), so it's safe to use, distribute, and embed freely in academic Java applications.

Language Level

The project makes use of 'SDK default' as our language level, which in our case is Java 21 due to the SDK we used. When developing or making changes to the project, it is highly recommended that the language level be configured to be language level 11 or higher, ensuring compatibility when working with GitLab, JavaFX and Temurin 21 our code remains well supported and stable.

JavaFX

The project relied heavily on JavaFX as our source of UI visuals; the version we used for this was 21, as can also be seen within our pom.xml. We chose this version due to it being the latest stable version available for use with Maven when we first developed our application. This was to ensure we benefited from the most up-to-date features, bug fixes, and performance improvements. JavaFX 21 also provided strong modular support and seamless integration with Java 21, making it a great choice for building our responsive application.

IntelliJ IDE

The project was developed using JetBrains' IntelliJ IDEA IDE (2024.2.5). When working on the university computers, we were under the Free Classroom and Free Educational licenses, provided to us by Aberystwyth University, as students; whilst, when working on private devices, each group member used their own licensed student subscription.

Maven

Using Maven to run the program gave us the freedom to avoid additional installation steps or complex setup processes, with just a single Maven command required to run the program. It also allowed us to configure the program folders so they wouldn't be overwritten when updating the project in the collaborative environment, which was particularly useful when uploading custom recordings, images and various other temporary files.

Junit

The project was tested using the unit testing framework from JUnit 5.10.0, specifically the JUnit Jupiter API module. This framework provides a powerful and flexible way of writing and executing unit tests in Java applications. It supports modern testing practices through annotations like `@Test`, `@Order`, and `@BeforeEach`, and integrates seamlessly with build tools like Maven, allowing tests to be automatically run during the application's build lifecycle. This ensures that code is consistently checked for correctness, reducing bugs and improving reliability throughout development.

2.2 Program Description

This program aimed to construct a flexible deck of cards application which allows a user to play any type of card game they desire. It has twelve primary functional requirements, as detailed in SE.QA.RS-CS22120 [1];

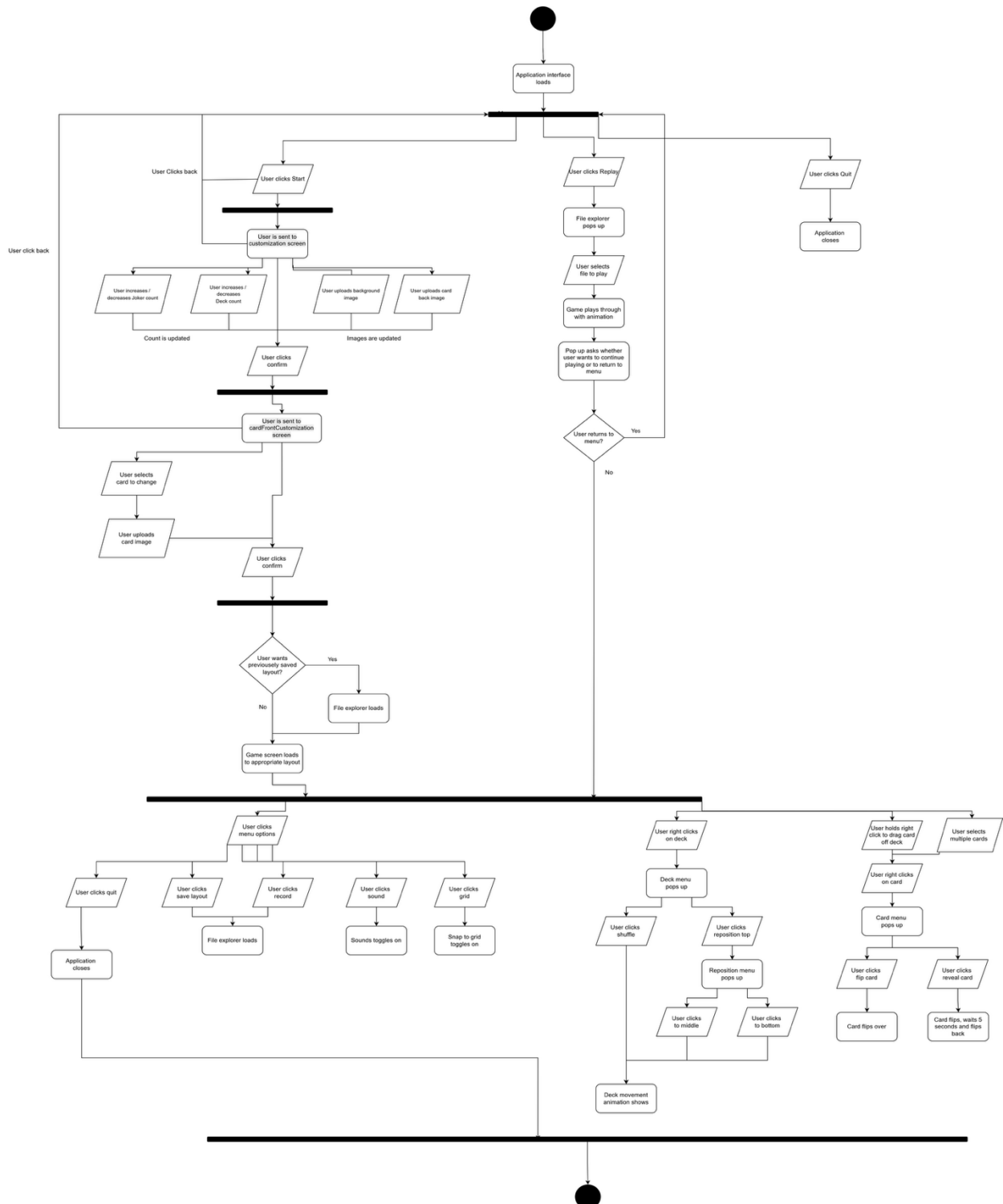
1. Define the number of decks on the table.
2. Define whether Jokers can be included and the number.
3. Define the images to be used for the cards.
4. Define the image used for the table background.
5. Be able to randomly shuffle a card deck.
6. Use the mouse/trackpad input to move a single card around the table.
7. Repositioning a card within a pack
8. Use the mouse/trackpad input to move multiple cards around the table.
9. Flip individual cards over.
10. Flip a group of cards over.
11. Reveal a group of cards.
12. To save the layout configuration of the cards.
13. To deal out the saved layout configuration.

Other additional and detailed functional requirements are detailed in SE.QA.RS-CS22120 [1]. The Design-Spec-GP6 document [8], section 4.1, describes the program logic of how the requirements were addressed, with specific references to classes and sequence diagrams within the same document.

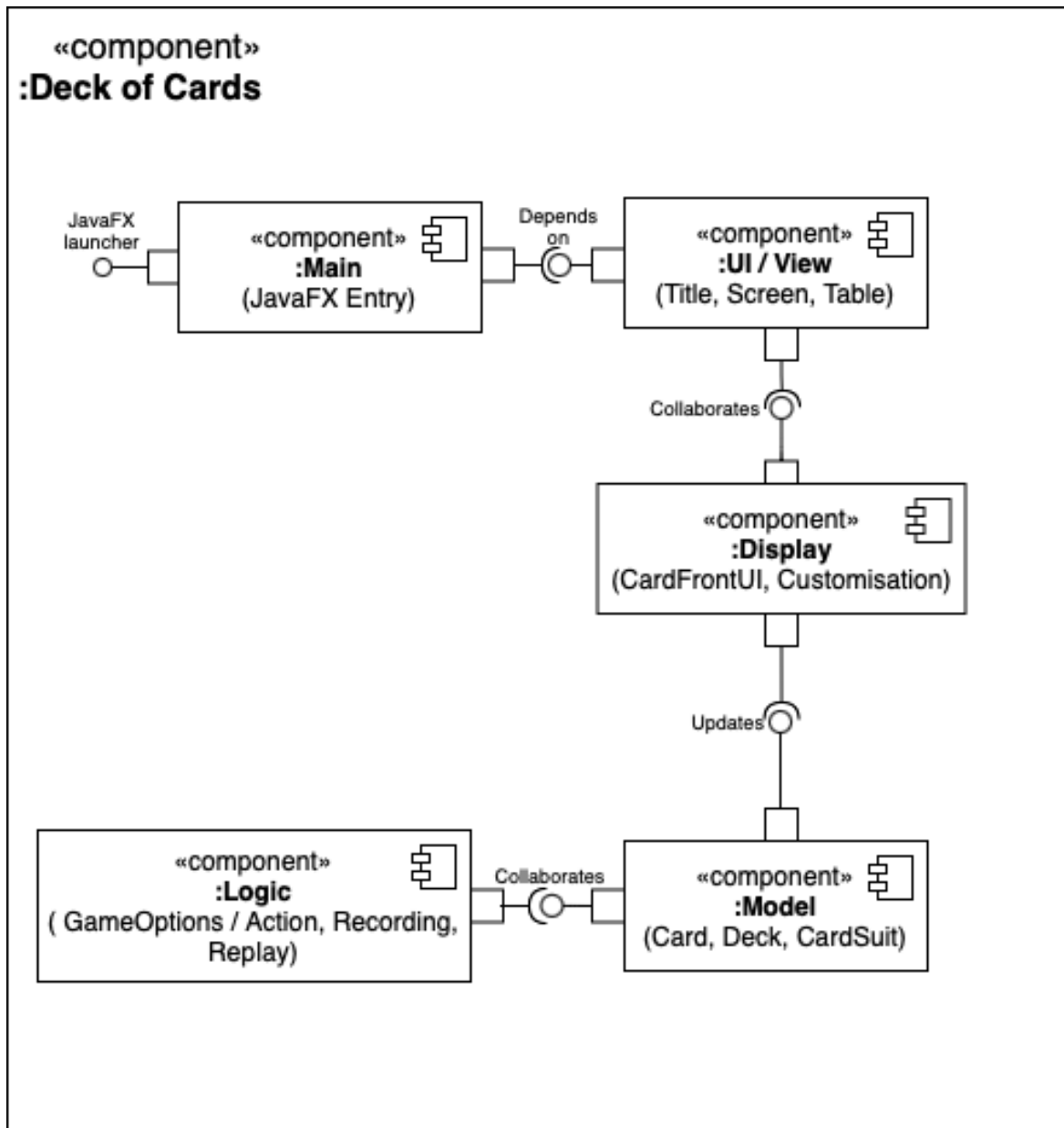
2.3 Program Structure

2.3.1 Flow of Control Diagram

Our program's State Diagram:



2.3.2 Component Diagram



2.3.3 Sequence Diagrams

Sequence diagrams can be found in section 4.1, under their respective user stories, of the Design-Spec-GP6 document [8].

2.3.4 UML Class Diagrams

The UML class diagrams can be found in section 5, subsection 5.3, of the Design-Spec-GP6 document [8].

2.4 Algorithms

Deck initialization and Shuffling

This program creates a standard 52-card deck and jokers depending on the game options: number of decks and jokers. The Algorithm used to shuffle the cards within the deck is the 'Java.util.Collections.shuffle' library function. This allows a randomised deck to simulate shuffling in real time game.

Card Drawing Logic

This program removes the first card from the deck list using `removeFirst()` to emulate drawing cards from a physical deck.

Creation of Cards

This program essentially uses a HashMap in the form *HashMap<Suit, Hashmap<Integer, Integer>>* as a frequency counter. This ensures that identical cards (same suit and index) have unique IDs for the replay logic.

Equality Checking

This program uses a method for checking if table objects are equal to detect changes during gameplay, this compares two decks based on content and order, not object identity

Recording and replay

This program handles recording, saving, loading and replaying game states. It converts deck and card states into strings, stores them in a list, writes them to or reads them from a file. During a replay, it reconstructs and displays each save state using helper methods from Deck and Card. It also includes comparison logic to verify if the current state matches a recorded one supporting the progress tracking.

2.5 Data Structures

Card

The Card class contains the graphical representation of a single card. It contains its value, suit and style properties. It consists of a nested map where each suit maps to another map that links a card's number to its associated integer values. The class works independently with the table logic and simply exists to hand its own logic and graphics components on screen, with each of its instances getting assigned various properties and functionalities that act as the card's interactive state.

Deck

The Deck class represents a single deck of 52 or more cards. And just as its real-life counterpart does, it contains all the independent cards it either began with or with the updated amount since the game was set to start. It consists of a List of Cards, as this best holds the needed objects in its most straightforward and convenient manner.

Replay / Recording

The game Recording and Replay systems use a combination of BlockingQueue and ArrayList structures to manage and store actions during gameplay. The lists used to store gameplay decisions and layout changes, respectively, are dynamic arrays that allow fast sequential access and efficient additions, making them well-suited for storing ordered game events.

Enums

The Suit and ActionType enums classes are fundamentals that help maintain the structure and clarity of the program's logic. The Suit enum defines the standard four suits of a card, providing a consistent and type-safe way to reference and compare card categories across the system. Meanwhile, the ActionType enum categorises all meaningful user or system interactions, such as flipping, drawing, or moving a card. It plays a key role in features like the replay and recording system we just talked about by allowing actions to be logged, tracked, and reconstructed in a structured format. Together, these enums enforce modular, reliable, and maintainable logic across components like Card, Replay, and Recording in the application.

2.6 Files

Card Visual Files

The CardFrontCustomization class is responsible for allowing users to personalise the appearance of cards by allowing the user to select the images for the card front's visuals. It handles file operations related to reading and saving these files. Specifically, it accesses image files from specified directories to display the last saved card face designs. When a new design is chosen, the class saves the preference to a settings file, stored in the project-level resources directory, ensuring that the chosen customisations persist between uses of the application.

The CardCustomization class also manages file interactions related to visual customisation. It handles the retrieval and saving of the images for both the game table and card backs. The card back's image is saved in the card directory within the project's resources, whilst the table background can be found directly in the root of the resources directory.

Recording Files

The Recording class captures a series of player actions, beginning from the moment the event is set in motion, and serialises them into a custom structured '.csgr' (Cloud Six Game Recording) file for the game recordings and '.csgrdl' (Cloud Six Game Recording Dealing Layout) for the dealing layout. The class writes these recordings to a user-chosen storage location, for easy retrieval and replay when necessary. Additionally, the class supports terminating active recording threads and safely closing file streams when the application exits, ensuring the integrity and consistency of saved data, as shown by the strict focus on the dialog box until the user decides to save or disregard the temporary data.

Replaying Files

The Replay class interacts with the recordings by reading saved gameplay files and reconstructing the actions in the given sequence. It loads the recorded action types and timestamps from the same directory passed by the user, then interprets and executes these in the application to visually replay the session. This class handles the interpretation of these recording files' timing and action consistency, helping developers or users to review gameplay history. It includes safeguards for invalid or malformed files and provides feedback if a file cannot be correctly read or executed.

2.7 Interfaces

In this program, there are three interfaces: tabletop, UI, and util.

For tabletop, this interface concerns the objects in which appear upon the table. This enables Cards and Decks the ability to inherit essential TableObject properties and methods, such as dragging, to make the gameplay more robust while keeping code as optimised as possible.

For UI, these are the visible elements that the players interact with in the game, such as the interaction through screens, buttons, menus and event handlers.

For util, these are the game options, game action and recording files that make the sure the program's infrastructure, repeated tasks and core mechanics are centralised, consistent and easy to manage. To allow for easier maintenance, it is separated from the primary game logic.

2.8 Suggestions for Improvements

Conserving card rotation after flipping/revealing

At present, when a card is rotated at any angle, flipping or revealing the card causes it to revert to its upright position. A potential improvement for this would be to create a method that uses a boolean attribute to check if the card has been rotated. If so, it then reinforces the card's saved rotation angle, maintaining its orientation after reveal or flip actions.

Disable the ability to drag cards off the screen

To prevent the cards from being dragged outside the visible Table area, are enforcing boundary checks during the drag event. In the TableObject file, we could use `Math.max()` and `Math.min()` to constrain the card's x and y coordinates, ensuring they stay within the Table's bounds. This ensures the cards are positioned within the scene boundaries and prevents them from moving off-screen.

Exiting full screen bug

A minor bug was identified during testing, primarily affecting observed Linux and MacOS. To mitigate the issues, we could add a JavaFX listener on the Table component. The listener would detect any state changes and trigger the UI updates when exiting full screen mode.

Sort out issues with replay

During replay, an error can be found that is caused by a very specific condition. While a card is in the process of being revealed, and is face-up, future actions will still be looking for its face-down version. Therefore, they aren't able to find the card it's acting on, which breaks the replay. The issue can be summarised as the system, when a user manually skips during a reveal action, is not able to have another action targeting the same card before the reveal animation ends. A fix for this can simply be to skip to the end of the animation manually, causing the action to finish immediately.

We discovered an issue when returning to the menu after completing a replay, and then starting a new one was that it caused the replay to end instantly. This could be fixed by ensuring that all the relevant state variables are fully reset before the next replay.

Developers' suggestions

Additional improvements could have included implementing 'nice-to-have' features described in the SE.QA.RS-CS22120 [1] document, along with perfecting aspects of implemented user stories like: 'resetting all card front images to their default files', or 'enabling the import of multiple image files simultaneously'.

If time had allowed it, we would have also refined small elements of our program's UI visuals, such as adding hover-state effects on buttons, to further enhance the user experience with subtle visual feedback.

2.9 Making Changes

The project is split primarily into three separate packages, Tabletop, UI and UTIL. As per the names, each contains parts relating to the game tabletop, game user interface and game utility. The contents of packages intersect throughout, so you must consider how the changes of one package may affect the other.

TableTop Package

Any changes to the TableObject.Java file will affect how both the Deck and Cards interact on the table. Know the recording file in the util package is effected by any possible alterations in how the objects are dragged as the movements are sent to a text file to be then replayed in the replay section

Furthermore, The Deck.Java file is important for the popup as it is a vital part of any game play. Changes made to initialisation, shuffling and placing a top card on any group, subtle bugs can easily appear as unexpected UI behaviour if not handled correctly.

UI Package

Any new changes made to the Popup.Java must be kept in mind that you will need to work in the tabletop package to properly implement into the game. However, this shouldn't be too much of a problem due to the structure of the pop-up code being easy to understand. Before implementing changes, review how the popup is invoked and handled within the tabletop package. Test all changes in a live game context to ensure that there is proper behaviour and flow.

UTIL Package

The Recording and Replay system function concurrently and rely on two separate but independent files. For the replay system to function correctly, the text file created must be present and complete. The recording processes the input file logs and reads this data in sequence to accurately reproduce past games. Ensure that no writes are interrupted during recording, as this may corrupt the file. If issues occur during replay, inspect both Recording.Java and Replay.java for why there could be missing data in the text file.

2.10 Physical Limitations

The Deck of Card game will run on most computers that have Java IntelliJ IDEA 2024 1.4 or later installed. The software is designed to run on faculty and IS desktops, it's performance on other systems has not been tested and cannot be confirmed.

2.11 Rebuilding and Testing

To rebuild the project, it is required that IntelliJ IDEA is installed. You can then use the following instructions:

Step 1: Obtain the source Code

The project source code can be obtained by either the following:

Using Git:

Open the terminal and execute the following command:

```
git clone git@gitlab.aber.ac.uk:nas97/gp6.git
```

Manually Downloading:

Navigate to <https://gitlab.aber.ac.uk/nas97/gp6/-/archive/main/gp6-main.zip>, download the ZIP file and extract its contents into a directory

Step 2: Open the project in IntelliJ

Launch IntelliJ IDEA, then open the extracted (or cloned project directory) and allow IntelliJ to index.

Step 3: Configure and Run

In the top-right corner of IntelliJ, find the configuration dropdown (left of the green run button) click edit configurations, then click the "+" button to add Maven. In the command line field, enter the command "*clean javafx:run -f pom.xml*" to then save and run the configuration.

Step 4: Run unit tests

To execute the Junit tests:

Repeat the configuration steps above; but in the command line field, enter "*clean test -f pom.xml*" and after completing these steps, the project should run and test successfully with IntelliJ.

3 References

- [1] SE.QA.RS-CS22120 - Deck of cards product backlog
- [2] SE.QA.04 - Ui prototype standards
- [3] SE.QA.05 - Design Standards
- [4] SE.QA.06 - Test Procedure Standards
- [5] SE.QA.08 - Configuration Management standards
- [6] SE.QA.09 - Coding standards
- [7] UI-Spec-Docu-GP6 - UI Specification Document and Presentation
- [8] Design-Spec-GP6 - Design Specification
- [9] Test-Spec-GP6 - Test Specification

4 Document History

Version	Issue Num.	Date	Changes made to document	Changes by
0.1	#226	01/05/25	Created the document	FS
0.2	#227	01/05/25	Completed the Introduction	FS
0.3	#230	02/05/25	Laid out References and Document history sections	FS
0.4	#228	05/05/25	Finished sections 2.1, 2.2, 2.3, 2.5 & 2.6	FS
0.5	#229	05/05/25	Finished sections 2.4, 2.7	NP
0.6	#231	06/05/25	Finished section 2.10, 2.11 and started section 2.8 and 2.9	NP
1.0	#231	06/05/25	Edited section 2.8	FS